
CP Reference Book

C++ Edition

Adán Arteaga

A contest-ready reference for competitive programming
and technical interviews

Every snippet compiles and runs as written — no pseudocode,
no placeholders. Complexities are stated where the choice matters.

Landscape layout, two columns, built for printing.
Clickable table of contents and full PDF bookmarks for on-screen use.

Contents

1	Contest Setup and Complexity	1
2	C++ STL Toolkit	2
3	Math, Modular Arithmetic and Number Theory	4
4	Arrays, Prefix Sums and Searching	7
5	Bit Manipulation	11
6	Strings	12
7	Graphs: Traversal and Ordering	14
8	Shortest Paths, MST and DSU	16
9	Trees and Advanced Graphs	18
10	Dynamic Programming	19
11	Advanced Data Structures	21
12	Contest Techniques and Pitfalls	22

1 Contest Setup and Complexity

Full Template

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using ull = unsigned long long;
using pii = pair<int,int>;
using pll = pair<ll,ll>;

#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (int)(x).size()

const ll INF = 4e18;
const int IINF = 1e9;
const ll MOD = 1e9 + 7;

void solve() {
    // ...
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t = 1;
    cin >> t;
    while (t--) solve();
    return 0;
}
```

Careful: `bits/stdc++.h` is GCC-only. On MSVC or Clang without it, include the headers you need explicitly.

Fast I/O

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
```

Careful: After this you cannot mix `cin` with `scanf`, or `cout` with `printf`. Pick one family and stay in it.

Tip: `'\n'` is faster than `endl`: `endl` flushes the buffer every single time. Only flush deliberately (interactive problems).

Decimal Precision

```
cout << fixed << setprecision(10) << ans << '\n';
```

Tip: Compare doubles with an epsilon, never with `==`: `fabs(a - b) < 1e-9`. When the answer is an integer computed in floating point, round with `llround(x)` instead of casting.

Data Type Limits

Type	Max	Precision
<code>int</code>	$\approx 2 \cdot 10^9$	—
<code>unsigned</code>	$\approx 4 \cdot 10^9$	—
<code>long long</code>	$\approx 9 \cdot 10^{18}$	—
<code>unsigned ll</code>	$\approx 1.8 \cdot 10^{19}$	—
<code>__int128</code>	$\approx 1.7 \cdot 10^{38}$	GCC only
<code>float</code>	—	7 digits
<code>double</code>	—	15–17 digits
<code>long double</code>	—	18–19 digits

Careful: The single most common bug in contests: `int * int` overflows *before* it is assigned to a `long long`. Cast one operand first: `(ll)a * b`.

Printing `__int128`

```
void print128(__int128 x) {
    if (x < 0) { putchar('-'); x = -x; }
    if (x > 9) print128(x / 10);
    putchar((char)('0' + (int)(x % 10)));
}
```

Use: exact products of two `long long` values, and safe modular multiplication when `MOD` exceeds 2^{31} .

Complexity Classes

- $O(1)$ — array index, hash lookup on average.
- $O(\log n)$ — binary search, `set`, heap push/pop.
- $O(\sqrt{n})$ — primality, divisor enumeration.
- $O(n)$ — a single pass.
- $O(n \log n)$ — sorting, most greedy solutions.
- $O(n^2)$ — nested loops, many DP tables.
- $O(2^n)$ — subsets, bitmask DP.
- $O(n!)$ — permutations.

Constraint to Complexity

Assume roughly 10^8 simple operations per second.

Constraint	Target
$n \leq 10$	$O(n!)$
$n \leq 20$	$O(2^n \cdot n)$
$n \leq 100$	$O(n^4)$
$n \leq 500$	$O(n^3)$
$n \leq 5000$	$O(n^2)$
$n \leq 10^5$	$O(n \log n)$
$n \leq 10^6$	$O(n)$ or $O(n \log n)$
$n \leq 10^{18}$	$O(\log n)$ or $O(\sqrt{n})$

Tip: Read the constraints *before* designing the solution. They usually name the intended complexity, which names the algorithm.

STL Complexity

vector

- `push_back`, `pop_back`, `[]` $\rightarrow O(1)$ amortized.
- `insert`, `erase` in the middle $\rightarrow O(n)$.

set / map / multiset (balanced BST)

- `insert`, `erase`, `find`, `lower_bound` $\rightarrow O(\log n)$.

unordered_map / unordered_set (hash table)

- `insert`, `find` $\rightarrow O(1)$ average, $O(n)$ worst case.

priority_queue (binary heap)

- `push`, `pop` $\rightarrow O(\log n)$; `top` $\rightarrow O(1)$.

deque

- `push_front`, `push_back`, `pop` on both ends $\rightarrow O(1)$.

Memory Budget

```
// 4 bytes each: 64M ints == 256 MB
int a[64 * 1024 * 1024];
```

Tip: Global arrays are zero-initialised and live in static storage; huge local arrays overflow the stack. Declare big arrays at global scope.

Compilation and Debugging

```
g++ -std=c++17 -O2 -Wall -Wextra sol.cpp -o sol
// Catches out-of-bounds and UB at runtime:
g++ -std=c++17 -g -fsanitize=address,undefined sol.cpp
```

```
// Debug print that vanishes on the judge
#ifdef LOCAL
#define dbg(x) cerr << #x << " = " << (x) << '\n'
#else
#define dbg(x)
#endif
```

Tip: Compile locally with `-DLOCAL`. `cerr` is not judged, so leftover debug output on `cerr` will not cause a wrong answer — but it does cost time.

2 C++ STL Toolkit

Vector Essentials

```
vector<int> v(n);           // n zeros
vector<int> v(n, -1);       // n copies of -1
vector<vector<int>> g(n, vector<int>(m, 0));

sort(all(v));
sort(rall(v));             // descending
reverse(all(v));

// Remove duplicates (must be sorted first)
sort(all(v));
v.erase(unique(all(v)), v.end());

int mn = *min_element(all(v));
int mx = *max_element(all(v));
ll s = accumulate(all(v), 0LL); // OLL avoids int overflow
int c = count(all(v), x);
```

Careful: `accumulate(all(v), 0)` sums into an `int`. Always pass `0LL` when the total can exceed $2 \cdot 10^9$.

Sorting with Custom Order

```
// By second value, then first descending
sort(all(v), [](const pii& a, const pii& b) {
    if (a.second != b.second) return a.second < b.second;
    return a.first > b.first;
});

// Sort indices instead of moving heavy objects
vector<int> idx(n);
iota(all(idx), 0);
sort(all(idx), [&](int i, int j) { return a[i] < a[j]; });
```

Careful: A comparator must be a *strict* weak ordering: it must return `false` for equal elements. Using `<=` is undefined behaviour and can segfault.

nth_element $O(n)$

```
// Places the k-th smallest at position k,
// partitions around it. No full sort.
```

```
nth_element(v.begin(), v.begin() + k, v.end());
int kth = v[k];
```

■ Pairs, Tuples and Bindings

```
pair<int,int> p = {1, 2};
tuple<int,int,int> t = {1, 2, 3};

auto [x, y] = p;           // C++17
auto [a, b, c] = t;

for (auto& [key, val] : mp) { // iterate a map cleanly
    // ...
}
```

■ set / map Operations

```
set<int> s;
s.insert(x);
s.erase(x);           // by value
s.erase(s.begin());   // by iterator

// First element >= x / first element > x
auto it = s.lower_bound(x);
auto jt = s.upper_bound(x);
if (it != s.end()) { /* *it exists */ }

// Largest element <= x
auto kt = s.upper_bound(x);
if (kt != s.begin()) { --kt; /* *kt <= x */ }
```

Careful: Use the *member* `s.lower_bound(x)`, not `std::lower_bound(all(s), x)`. The free function on a **set** degrades to $O(n)$ because the iterators are not random access.

```
map<int,int> mp;
mp[k]++;               // inserts 0 then increments
if (mp.count(k)) { /* ... */ }
mp.erase(k);

// Does NOT insert when missing
auto it = mp.find(k);
if (it != mp.end()) use(it->second);
```

Careful: `mp[k]` default-constructs a missing key. Inside a read-only loop that silently grows the map; use `find` or `count`.

■ multiset: Erase One vs All

```
multiset<int> ms;
ms.erase(x);           // erases EVERY copy of x
ms.erase(ms.find(x));  // erases exactly one
```

■ priority_queue

```
priority_queue<int> maxHeap; // largest on top (default)
```

```
priority_queue<int, vector<int>, greater<int>> minHeap;

// Min-heap of pairs, the common Dijkstra shape
priority_queue<pll, vector<pll>, greater<pll>> pq;
pq.push({dist, node});
```

Tip: No **greater** handy? Push negated values into a max-heap and negate on the way out.

■ deque

```
deque<int> dq;
dq.push_back(x); dq.push_front(x);
dq.pop_back(); dq.pop_front();
int f = dq.front(), b = dq.back();
```

Use: sliding window extrema and 0–1 BFS.

■ bitset

```
bitset<1000> b;
b[i] = 1;
b.set(); b.reset(); b.flip();
int ones = b.count();
bool any = b.any();

// Set operations run 64 bits at a time
bitset<1000> c = a & b;
c = a | b; c = a ^ b; c = a << 3;
```

Use: a constant-factor $1/64$ speedup. It turns an $O(n^2)$ DP or reachability computation into $O(n^2/64)$, which is often enough to pass.

■ next_permutation

```
sort(all(v));           // must start sorted
do {
    // process this permutation
} while (next_permutation(all(v)));
```

■ Anti-Hash Custom Hash

```
struct chash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t SEED =
            chrono::steady_clock::now()
                .time_since_epoch().count();
        return splitmix64(x + SEED);
    }
};

unordered_map<ll, int, chash> mp;
```

```
mp.reserve(1 << 20);
mp.max_load_factor(0.25);
```

Careful: The default `unordered_map` hash for integers is the identity. On Codeforces, anyone can feed you colliding keys and turn your $O(1)$ lookups into $O(n)$. Use this on any open hacking round, or just use `map`.

Randomness

```
mt19937_64 rng(chrono::steady_clock::now()
               .time_since_epoch().count());
```

```
ll r = rng() % n; // biased but usually fine
ll u = uniform_int_distribution<ll>(1, n)(rng);
shuffle(all(v), rng);
```

Careful: Never seed with a constant, and never use `rand()` — on Windows it only produces values up to 32767.

3 Math, Modular Arithmetic and Number Theory

GCD and LCM $\mathcal{O}(\log \min(a, b))$

```
// GCC built-in, needs <algorithm>
int g = __gcd(a, b);

ll gcd_it(ll a, ll b) {
    while (b) { a %= b; swap(a, b); }
    return a < 0 ? -a : a;
}

ll lcm_safe(ll a, ll b) {
    if (a == 0 || b == 0) return 0;
    return (a / __gcd(a, b)) * b; // divide FIRST
}
```

Properties:

- $\gcd(a, b) \cdot \text{lcm}(a, b) = |a \cdot b|$.
- $\gcd(a, b) = \gcd(b, a \bmod b)$.
- $\gcd(a, b, c) = \gcd(a, \gcd(b, c))$.

Careful: In `lcm`, dividing before multiplying is what keeps the product from overflowing.

Extended Euclid $\mathcal{O}(\log \min(a, b))$

Solves $ax + by = \gcd(a, b)$.

```
ll extgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1;
    ll g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}
```

Use: modular inverse when `MOD` is *not* prime, and linear Diophantine equations.

Linear Diophantine

$ax + by = c$ has a solution iff $\gcd(a, b) \mid c$.

```
bool solve_dio(ll a, ll b, ll c, ll& x, ll& y) {
    ll g = extgcd(a, b, x, y);
    if (c % g != 0) return false;
    x *= c / g;
    y *= c / g;
    return true;
}
// General family:
// x + k*(b/g), y - k*(a/g) for any integer k
```

Modulo Properties

$$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$

$$(a - b) \bmod m = ((a \bmod m) - (b \bmod m) + m) \bmod m$$

$$(a \cdot b) \bmod m = ((a \bmod m)(b \bmod m)) \bmod m$$

```
// C++ % keeps the sign of the dividend
int r = (a % MOD + MOD) % MOD;
```

Careful: Division does *not* distribute over the modulus. Multiply by the modular inverse instead.

Binary Exponentiation $\mathcal{O}(\log b)$

```
ll binpow(ll a, ll b, ll mod) {
    ll res = 1;
    a %= mod;
    if (a < 0) a += mod;
    while (b > 0) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
```

Safe Multiplication mod m

```
// Needed when mod > 2^31 (a*b overflows ll)
ll mulmod(ll a, ll b, ll mod) {
```

```
    return (__int128)a * b % mod;
}
```

■ Modular Inverse $\mathcal{O}(\log m)$

```
// MOD prime, a not divisible by MOD (Fermat)
ll inv(ll a, ll mod) { return binpow(a, mod - 2, mod); }

// Any mod, requires gcd(a, mod) == 1
ll inv_gen(ll a, ll mod) {
    ll x, y;
    ll g = extgcd(a, mod, x, y);
    if (g != 1) return -1;    // no inverse
    return (x % mod + mod) % mod;
}
```

■ All Inverses 1..n $\mathcal{O}(n)$

```
vector<ll> inv(n + 1);
inv[1] = 1;
for (int i = 2; i <= n; i++)
    inv[i] = (MOD - (MOD / i) * inv[MOD % i] % MOD) % MOD;
```

Use: one linear pass instead of n separate $\mathcal{O}(\log m)$ exponentiations.

■ Factorials and nCr $\mathcal{O}(n \text{ precompute, } 1 \text{ per query})$

```
const int MAXN = 1e6 + 5;
ll fact[MAXN], invfact[MAXN];

void init_fact() {
    fact[0] = 1;
    for (int i = 1; i < MAXN; i++)
        fact[i] = fact[i - 1] * i % MOD;

    invfact[MAXN - 1] = binpow(fact[MAXN - 1], MOD - 2, MOD);
    for (int i = MAXN - 1; i > 0; i--)
        invfact[i - 1] = invfact[i] * i % MOD;
}

ll nCr(ll n, ll r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD;
}

ll nPr(ll n, ll r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invfact[n - r] % MOD;
}
```

$$C(n, r) = \frac{n!}{r!(n-r)!} \quad P(n, r) = \frac{n!}{(n-r)!}$$

Tip: The backward loop computes every inverse factorial with a single exponentiation. Calling `binpow` inside `nCr` is the slow way and shows up as TLE with many queries.

■ Combinatorial Identities

- $C(n, r) = C(n - 1, r - 1) + C(n - 1, r)$ — Pascal.
- $\sum_{r=0}^n C(n, r) = 2^n$.
- Stars and bars: $x_1 + \dots + x_k = n$, $x_i \geq 0 \rightarrow C(n + k - 1, k - 1)$.
- Catalan: $C_n = \frac{1}{n+1} \binom{2n}{n}$ — balanced brackets, binary trees, triangulations.

■ Sieve of Eratosthenes $\mathcal{O}(n \log \log n)$

```
const int MAXN = 1e6 + 5;
vector<bool> isPrime(MAXN, true);
vector<int> primes;

void sieve(int n) {
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i <= n; i++) {
        if (isPrime[i]) {
            primes.push_back(i);
            for (ll j = (ll)i * i; j <= n; j += i)
                isPrime[j] = false;
        }
    }
}
```

Careful: Start the inner loop at i^2 and use `ll` for `j`: $i \cdot i$ overflows `int` once $i > 46340$.

Linear Sieve with SPF $\mathcal{O}(n)$

```
vector<int> spf(MAXN, 0), primes;

void linear_sieve(int n) {
    for (int i = 2; i <= n; i++) {
        if (spf[i] == 0) {
            spf[i] = i;
            primes.push_back(i);
        }
        for (int p : primes) {
            if (p > spf[i] || (ll)i * p > n) break;
            spf[i * p] = p;
        }
    }

    // O(log x) factorization after the sieve
    vector<int> factorize_spf(int x) {
        vector<int> f;
        while (x > 1) { f.push_back(spf[x]); x /= spf[x]; }
        return f;
    }
}
```

Use: when you must factor many numbers. One $\mathcal{O}(n)$ precomputation turns each factorization into $\mathcal{O}(\log x)$.

Trial Division Factorization $\mathcal{O}(\sqrt{n})$

```
vector<pll> factorize(ll x) {
    vector<pll> f; // {prime, exponent}
    for (ll p = 2; p * p <= x; p++) {
        if (x % p == 0) {
            int e = 0;
            while (x % p == 0) { x /= p; e++; }
            f.push_back({p, e});
        }
    }
    if (x > 1) f.push_back({x, 1});
    return f;
}
```

All Divisors $\mathcal{O}(\sqrt{n})$

```
vector<ll> divisors(ll n) {
    vector<ll> d;
    for (ll i = 1; i * i <= n; i++) {
        if (n % i == 0) {
            d.push_back(i);
            if (i != n / i) d.push_back(n / i);
        }
    }
    sort(all(d));
    return d;
}
```

From the factorization $n = \prod p_i^{e_i}$:

- Number of divisors: $\prod (e_i + 1)$.

- Sum of divisors: $\prod \frac{p_i^{e_i+1} - 1}{p_i - 1}$.

Divisor Sieve $\mathcal{O}(n \log n)$

```
// divs[i] = list of all divisors of i, for every i <= n
vector<vector<int>> divs(n + 1);
for (int i = 1; i <= n; i++)
    for (int j = i; j <= n; j += i)
        divs[j].push_back(i);
```

Euler's Totient Function

$\phi(n)$ counts integers in $[1, n]$ coprime to n .

```
// Single value, O(sqrt n)
ll phi(ll n) {
    ll res = n;
    for (ll p = 2; p * p <= n; p++) {
        if (n % p == 0) {
            while (n % p == 0) n /= p;
            res -= res / p;
        }
    }
    if (n > 1) res -= res / n;
    return res;
}

// Sieve for all i <= n, O(n log log n)
vector<int> phi_all(int n) {
    vector<int> f(n + 1);
    iota(f.begin(), f.end(), 0);
    for (int i = 2; i <= n; i++)
        if (f[i] == i) // i is prime
            for (int j = i; j <= n; j += i)
                f[j] -= f[j] / i;
    return f;
}
```

Use: Euler's theorem: $a^{\phi(m)} \equiv 1 \pmod{m}$ when $\gcd(a, m) = 1$. This is how you reduce huge exponents modulo a composite m .

Miller-Rabin Primality $\mathcal{O}(k \log^3 n)$

```
bool check_composite(ull n, ull a, ull d, int s) {
    ull x = binpow(a, d, n);
    if (x == 1 || x == n - 1) return false;
    for (int r = 1; r < s; r++) {
        x = (__int128)x * x % n;
        if (x == n - 1) return false;
    }
    return true;
}

bool is_prime(ull n) {
    if (n < 2) return false;
    int s = 0;
    ull d = n - 1;
    while ((d & 1) == 0) { d >>= 1; s++; }
}
```

```
// Deterministic for every 64-bit n
for (ull a : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL,
             17ULL, 19ULL, 23ULL, 29ULL, 31ULL, 37ULL}) {
    if (n == a) return true;
    if (check_composite(n, a, d, s)) return false;
}
return true;
}
```

Use: primality of values up to 10^{18} , where trial division is hopeless. Requires the `__int128` binpow.

■ Pollard's Rho Factorization $\mathcal{O}(n^{1/4})$

```
ull pollard(ull n) {
    if (n % 2 == 0) return 2;
    ull x = rng() % n, y = x, c = rng() % n + 1, d = 1;
    while (d == 1) {
        x = ((__int128)x * x + c) % n;
        y = ((__int128)y * y + c) % n;
        y = ((__int128)y * y + c) % n;
        d = __gcd(x > y ? x - y : y - x, n);
    }
    return d == n ? pollard(n) : d;
}

void factor(ull n, vector<ull>& out) {
    if (n == 1) return;
    if (is_prime(n)) { out.push_back(n); return; }
    ull d = pollard(n);
    factor(d, out);
    factor(n / d, out);
}
```

Use: factoring numbers up to 10^{18} in milliseconds.

■ Chinese Remainder Theorem $\mathcal{O}(\log m)$

Merges $x \equiv a_1 \pmod{m_1}$ and $x \equiv a_2 \pmod{m_2}$.

```
// Returns {x, lcm} or {-1, -1} when incompatible
pll crt(ll a1, ll m1, ll a2, ll m2) {
    ll p, q;
    ll g = extgcd(m1, m2, p, q);
    if ((a2 - a1) % g != 0) return {-1, -1};

    ll lcm = m1 / g * m2;
    __int128 t = (__int128)(a2 - a1) / g % (m2 / g) * p % (m2 / g);
    ll x = (ll)((__int128)a1 + (__int128)t * m1 % lcm);
    return {(x % lcm + lcm) % lcm, lcm};
}
```

4 Arrays, Prefix Sums and Searching

■ Prefix Sum 1D $\mathcal{O}(n \text{ build, } 1 \text{ query})$

```
// pref[i] = sum of the first i elements
vector<ll> pref(n + 1, 0);
```

```
}
```

■ Matrix Exponentiation $\mathcal{O}(k^3 \log n)$

```
using Mat = vector<vector<ll>>>;

Mat mul(const Mat& A, const Mat& B) {
    int n = A.size(), m = B[0].size(), k = B.size();
    Mat C(n, vector<ll>(m, 0));
    for (int i = 0; i < n; i++)
        for (int p = 0; p < k; p++) {
            if (!A[i][p]) continue;
            for (int j = 0; j < m; j++)
                C[i][j] = (C[i][j] + A[i][p] * B[p][j]) % MOD;
        }
    return C;
}

Mat matpow(Mat A, ll e) {
    int n = A.size();
    Mat R(n, vector<ll>(n, 0));
    for (int i = 0; i < n; i++) R[i][i] = 1;
    while (e) {
        if (e & 1) R = mul(R, A);
        A = mul(A, A);
        e >>= 1;
    }
    return R;
}
```

Use: any linear recurrence at a huge index. Fibonacci: $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$ gives F_n in $\mathcal{O}(\log n)$. Also counts walks of length n in a graph.

■ Notable Sums

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2}\right)^2 \quad \sum_{i=0}^n r^i = \frac{r^{n+1} - 1}{r - 1}$$

```
ll s1 = n * (n + 1) / 2;
ll s2 = n * (n + 1) * (2 * n + 1) / 6;
```

Careful: Compute in `ll`: for $n = 10^6$, $n(n+1)$ already overflows `int`.

```
for (int i = 0; i < n; i++)
    pref[i + 1] = pref[i] + a[i];

// Inclusive range [l, r], 0-indexed
ll range_sum(int l, int r) { return pref[r + 1] - pref[l]; }
```

Tip: Sizing the array $n + 1$ and shifting by one removes every $1 == 0$ special case.

■ Prefix XOR

```
vector<int> px(n + 1, 0);  
for (int i = 0; i < n; i++) px[i + 1] = px[i] ^ a[i];  
  
int range_xor(int l, int r) { return px[r + 1] ^ px[l]; }
```

Use: XOR is its own inverse, so subtraction becomes another XOR.

■ Prefix Sum 2D $\mathcal{O}(nm \text{ build, } 1 \text{ query})$

```
vector<vector<ll>> p(n + 1, vector<ll>(m + 1, 0));
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        p[i][j] = a[i-1][j-1] + p[i-1][j]
            + p[i][j-1] - p[i-1][j-1];

// Inclusive rectangle, 0-indexed corners
ll rect(int x1, int y1, int x2, int y2) {
    x1++; y1++; x2++; y2++;
    return p[x2][y2] - p[x1-1][y2]
        - p[x2][y1-1] + p[x1-1][y1-1];
}
```

■ Difference Array 1D $\mathcal{O}(1 \text{ per update})$

```
vector<ll> d(n + 1, 0);

void add_range(int l, int r, ll v) { // inclusive
    d[l] += v;
    if (r + 1 <= n) d[r + 1] -= v;
}

// Rebuild the final array once, at the end
ll cur = 0;
for (int i = 0; i < n; i++) { cur += d[i]; a[i] += cur; }
```

Use: many range updates and a single read at the end. It is the exact mirror of a prefix sum.

■ Difference Array 2D

```
vector<vector<ll>> d(n + 2, vector<ll>(m + 2, 0));

void add_rect(int x1, int y1, int x2, int y2, ll v) {
    d[x1][y1] += v;
    d[x1][y2+1] -= v;
    d[x2+1][y1] -= v;
    d[x2+1][y2+1] += v;
}

// Then take a 2D prefix sum of d to get the final grid
```

■ Binary Search $\mathcal{O}(\log n)$

```
// Classic: find x in a sorted array
int l = 0, r = n - 1, ans = -1;
while (l <= r) {
    int mid = l + (r - 1) / 2; // avoids overflow
    if (a[mid] == x) { ans = mid; break; }
    if (a[mid] < x) l = mid + 1;
    else r = mid - 1;
}
```

Careful: $(l + r) / 2$ overflows when $l + r$ exceeds `INT_MAX`. $l + (r - 1) / 2$ never does.

■ Binary Search on the Answer $\mathcal{O}(\log(\text{range}) \cdot f)$

```
// Smallest value that satisfies a monotone predicate
ll lo = 0, hi = 1e18, ans = -1;
while (lo <= hi) {
    ll mid = lo + (hi - lo) / 2;
    if (check(mid)) { ans = mid; hi = mid - 1; }
    else { lo = mid + 1; }
}
```

Use: "minimise the maximum", "maximise the minimum", and any question whose answer is monotone: if `mid` works then everything above (or below) works too.

■ First True Pattern

```
// Half-open [lo, hi): no ans variable needed
int lo = 0, hi = n;
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (ok(mid)) hi = mid;
    else lo = mid + 1;
}
// lo == first index where ok() is true (or n)
```

Tip: This shape is much harder to get wrong than the `l <= r` version. Learn one and reuse it everywhere.

■ Binary Search on Doubles

```
double lo = 0, hi = 1e9;
for (int iter = 0; iter < 100; iter++) { // fixed count
    double mid = (lo + hi) / 2;
    if (check(mid)) hi = mid;
    else lo = mid;
}
```

Tip: Iterate a fixed number of times rather than comparing against an epsilon. 100 halvings is far below any precision you will be asked for, and it cannot loop forever.

■ lower_bound / upper_bound $\mathcal{O}(\log n)$

```
// lower_bound: first element >= x
auto it = lower_bound(all(v), x);
int idx = it - v.begin();
if (it == v.end()) { /* nothing >= x */ }

// upper_bound: first element > x
auto jt = upper_bound(all(v), x);

// Count of x
int cnt = upper_bound(all(v), x) - lower_bound(all(v), x);

// Largest element <= x
auto kt = upper_bound(all(v), x);
if (kt != v.begin()) { --kt; /* *kt <= x */ }
```

Careful: Both require the array to be sorted. On unsorted input they return garbage silently.

■ Ternary Search $\mathcal{O}(\log(\text{range}))$

```
// Maximum of a strictly unimodal function
double lo = 0, hi = 1e9;
for (int iter = 0; iter < 200; iter++) {
    double m1 = lo + (hi - lo) / 3;
    double m2 = hi - (hi - lo) / 3;
    if (f(m1) < f(m2)) lo = m1;
    else hi = m2;
}
double best = f(lo);
```

Use: unimodal functions — one peak, no plateau. For a minimum, flip the comparison.

■ Two Pointers $\mathcal{O}(n)$

```
int l = 0;
ll sum = 0;
for (int r = 0; r < n; r++) {
    sum += a[r];           // extend the window
    while (sum > K) {      // shrink while invalid
        sum -= a[l];
        l++;
    }
    // [l, r] is now the longest valid window ending at r
    best = max(best, r - l + 1);
}
```

Careful: This only works when the predicate is monotone in the window: once a window is invalid, extending it right cannot make it valid again.

■ Sliding Window Maximum $\mathcal{O}(n)$

```
// Max of every window of size k
deque<int> dq;           // stores indices
vector<int> res;
for (int i = 0; i < n; i++) {
    while (!dq.empty() && dq.front() <= i - k)
        dq.pop_front();    // drop out-of-window
    while (!dq.empty() && a[dq.back()] <= a[i])
        dq.pop_back();     // drop dominated
    dq.push_back(i);
    if (i >= k - 1) res.push_back(a[dq.front()]);
}
```

Tip: The deque stays decreasing, so the front is always the window maximum. Flip the comparison for a minimum.

■ Kadane: Max Subarray Sum $\mathcal{O}(n)$

```
ll best = LLONG_MIN, cur = 0;
for (int i = 0; i < n; i++) {
    cur = max((ll)a[i], cur + a[i]);
    best = max(best, cur);
}
```

Careful: Starting **best** at 0 breaks on all-negative arrays. Start it at **LLONG_MIN** and let the

first element set it.

■ LIS $\mathcal{O}(n \log n)$

```
// Length only
int lis_length(const vector<int>& a) {
    vector<int> t;
    for (int x : a) {
        auto it = lower_bound(all(t), x);
        if (it == t.end()) t.push_back(x);
        else *it = x;
    }
    return sz(t);
}
```

Careful: **t** is *not* the LIS itself, only its length is meaningful. Reconstruct with parent pointers:

```
vector<int> lis_reconstruct(const vector<int>& a) {
    int n = sz(a);
    vector<int> t, pos, par(n, -1);
    for (int i = 0; i < n; i++) {
        int j = lower_bound(all(t), a[i]) - t.begin();
        if (j > 0) par[i] = pos[j - 1];
        if (j == sz(t)) { t.push_back(a[i]); pos.push_back(i); }
        else { t[j] = a[i]; pos[j] = i; }
    }
    vector<int> res;
    for (int i = pos.back(); i != -1; i = par[i])
        res.push_back(a[i]);
    reverse(all(res));
    return res;
}
```

Tip: Swap **lower_bound** for **upper_bound** to get the longest *non-decreasing* subsequence.

■ Coordinate Compression $\mathcal{O}(n \log n)$

```
vector<int> vals = a;
sort(all(vals));
vals.erase(unique(all(vals)), vals.end());

for (int& x : a)
    x = lower_bound(all(vals), x) - vals.begin();
// vals[x] recovers the original value
```

Use: values up to 10^9 but only $n \leq 10^5$ of them — compress to $[0, n)$ so they can index a BIT or segment tree.

■ Counting Inversions $\mathcal{O}(n \log n)$

```
// Pairs i < j with a[i] > a[j], via merge sort
ll merge_count(vector<int>& a, int l, int r) {
    if (r - l <= 1) return 0;
    int m = (l + r) / 2;
    ll cnt = merge_count(a, l, m) + merge_count(a, m, r);

    vector<int> tmp;
    int i = l, j = m;
    while (i < m && j < r) {
        if (a[i] <= a[j]) tmp.push_back(a[i++]);
    }
```

```

    else { cnt += m - i; tmp.push_back(a[j++]); }
}
while (i < m) tmp.push_back(a[i++]);
while (j < r) tmp.push_back(a[j++]);
copy(all(tmp), a.begin() + 1);
return cnt;
}

```

Use: the minimum number of adjacent swaps that sorts the array.

Monotonic Stack $\mathcal{O}(n)$

```

// Next strictly greater element to the right
vector<int> nxt(n, -1);
stack<int> st; // indices, decreasing values
for (int i = 0; i < n; i++) {
    while (!st.empty() && a[st.top()] < a[i]) {
        nxt[st.top()] = i;
        st.pop();
    }
    st.push(i);
}

```

Use: previous/next greater or smaller, largest rectangle in a histogram, and the classic "sum of subarray minimums".

5 Bit Manipulation

Basic Operations

```

mask | (1LL << i) // set bit i
mask & (1LL << i) // test bit i (nonzero if set)
mask ^ (1LL << i) // toggle bit i
mask & ~(1LL << i) // clear bit i

(mask >> i) & 1 // test, as a clean 0/1

```

Careful: $1 \ll i$ is an `int`. For $i \geq 31$ this is undefined behaviour — write `1LL << i` whenever the mask can be wide.

Least Significant Bit

```

mask & -mask // isolate the lowest set bit
mask & (mask - 1) // clear the lowest set bit
mask | (mask + 1) // set the lowest clear bit

```

GCC Built-ins

```

__builtin_popcount(x) // number of set bits (int)
__builtin_popcountll(x) // ... (long long)
__builtin_ctz(x) // trailing zeros -> index of LSB
__builtin_clz(x) // leading zeros
__builtin_ffs(x) // 1-based index of LSB, 0 if x == 0
__builtin_parity(x) // popcount % 2

```

Careful: `ctz` and `clz` are undefined for `x == 0`. Guard them.

Tip: C++20 offers the portable `<bit>` header: `popcount`, `countr_zero`, `countl_zero`, `bit_width`, `has_single_bit`.

Common Tricks

```

(1LL << n) - 1 // n low bits all set
n > 0 && !(n & (n - 1)) // is a power of two
31 - __builtin_clz(n) // floor(log2(n)), int

```

```

63 - __builtin_clzll(n) // floor(log2(n)), long long
1 << (32 - __builtin_clz(n - 1)) // next power of two
x & 1 // odd?
x >> 1 // x / 2 for non-negative x

```

Careful: Right-shifting a *negative* number is implementation-defined rounding, not division. Use `/ 2` when the sign matters.

XOR Properties

- $a \oplus a = 0$ and $a \oplus 0 = a$.
- Commutative and associative — order never matters.
- If $a \oplus b = c$ then $a \oplus c = b$.
- XOR of $1..n$ has period 4: $n, 1, n+1, 0$ for $n \bmod 4 = 0, 1, 2, 3$.

Use: "every value appears twice except one" — XOR everything, the pairs cancel out.

Iterate All Subsets $\mathcal{O}(2^n \cdot n)$

```

for (int mask = 0; mask < (1 << n); mask++) {
    for (int j = 0; j < n; j++) {
        if (mask & (1 << j)) {
            // element j belongs to this subset
        }
    }
}

```

Iterate Submasks $\mathcal{O}(3^n \text{ total})$

```

for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
    // sub is a nonempty submask of mask
}
// Include the empty submask:
int sub = mask;
while (true) {
    // process sub
}

```

```

    if (sub == 0) break;
    sub = (sub - 1) & mask;
}

```

Tip: Summed over every `mask`, this loop runs 3^n times, not 4^n — which is what makes subset-sum DP feasible for $n \leq 20$.

Subsets of Size K (Gosper's Hack) $\mathcal{O}\left(\binom{n}{k}\right)$

```

int cur = (1 << k) - 1;
while (cur < (1 << n)) {
    // process cur: exactly k bits set
    int c = cur & -cur;
    int r = cur + c;
    cur = ((r ^ cur) >> 2) / c | r;
}

```

SOS DP (Sum over Subsets) $\mathcal{O}(2^n \cdot n)$

```

// f[mask] becomes the sum of a[sub] over all sub of mask
vector<ll> f(1 << n);
for (int mask = 0; mask < (1 << n); mask++) f[mask] = a[mask];

for (int i = 0; i < n; i++)
    for (int mask = 0; mask < (1 << n); mask++)
        if (mask & (1 << i))
            f[mask] += f[mask ^ (1 << i)];

```

Use: aggregating over all 2^n submasks in $\mathcal{O}(2^n n)$ instead of $\mathcal{O}(3^n)$. Reverse the `if` to sum over supersets.

XOR Linear Basis $\mathcal{O}(n \log C)$

```

// Maximal XOR of any subset of the inserted values
struct Basis {
    static const int B = 60;
    ll b[B + 1] = {};

    bool insert(ll x) {
        for (int i = B; i >= 0; i--) {
            if (!((x >> i) & 1)) continue;
            if (!b[i]) { b[i] = x; return true; }
            x ^= b[i];
        }
        return false; // x was representable
    }

    ll max_xor() {
        ll res = 0;
        for (int i = B; i >= 0; i--)
            res = max(res, res ^ b[i]);
        return res;
    }
};

```

Use: "maximum XOR of a subset", counting distinct achievable XOR values (2^{rank}), and checking whether a value is reachable.

6 Strings

Reading Input

```

string s;
getline(cin, s); // whole line, spaces included

```

Careful: After `cin >> n`, the newline is still in the buffer and the next `getline` reads an empty string. Clear it first:

```

int n;
cin >> n;
cin.ignore(); // discard the pending newline
string s;
getline(cin, s);

```

Tokenising with stringstream

```

string line = "10 20 30";
stringstream ss(line);
int a, b, c;
ss >> a >> b >> c;

// Unknown count of words
vector<string> words;

```

```

string w;
while (ss >> w) words.push_back(w);

// Split on a custom delimiter
while (getline(ss, w, ',')) words.push_back(w);

```

String Utilities

```

s.substr(pos, len); // len is a COUNT, not an index
s.find("abc"); // string::npos if absent
s += 'x';
reverse(all(s));
sort(all(s)); // anagram normal form

int digit = s[i] - '0';
char up = toupper(s[i]);
bool isLetter = isalpha(s[i]);

string t = to_string(42);
int v = stoi(t);
ll w = stoll(t);

```

Careful: `s.substr(l, r)` does *not* take a right index. For the inclusive range $[l, r]$ write `s.substr(l, r - l + 1)`.

■ Character Frequency

```
array<int, 26> cnt{};
for (char c : s) cnt[c - 'a']++;

// Anagram test
array<int, 26> ca{}, cb{};
for (char c : a) ca[c - 'a']++;
for (char c : b) cb[c - 'a']++;
bool anagram = (ca == cb);
```

■ Z-Function $\mathcal{O}(n)$

$z[i]$ = length of the longest common prefix of s and $s[i..]$.

```
vector<int> z_function(const string& s) {
    int n = sz(s);
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
    }
    return z;
}

vector<int> z_search(const string& text, const string& pat) {
    string s = pat + '#' + text; // '#' must not occur
    vector<int> z = z_function(s), res;
    int m = sz(pat);
    for (int i = m + 1; i < sz(s); i++)
        if (z[i] == m) res.push_back(i - m - 1);
    return res;
}
```

Careful: The separator must be a character that appears in neither string, or matches can straddle the join.

■ KMP Prefix Function $\mathcal{O}(n)$

$pi[i]$ = length of the longest proper prefix of $s[0..i]$ that is also a suffix of it.

```
vector<int> prefix_function(const string& s) {
    int n = sz(s);
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}

vector<int> kmp_search(const string& text, const string& pat) {
    string s = pat + '#' + text;
    vector<int> pi = prefix_function(s), res;
    int m = sz(pat);
    for (int i = m + 1; i < sz(s); i++)
        if (pi[i] == m) res.push_back(i - 2 * m);
}
```

```
return res;
}
```

Use: $n - pi[n-1]$ is the smallest period of the string. It repeats exactly when that period divides n .

■ Manacher $\mathcal{O}(n)$

```
// d1[i] = radius of the odd palindrome centred at i
vector<int> manacher_odd(const string& s) {
    int n = sz(s);
    vector<int> d1(n);
    int l = 0, r = -1;
    for (int i = 0; i < n; i++) {
        int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
        while (0 <= i - k && i + k < n && s[i - k] == s[i + k]) k++;
        d1[i] = k;
        if (i + k - 1 > r) { l = i - k + 1; r = i + k - 1; }
    }
    return d1;
}

// d2[i] = radius of the even palindrome between i-1 and i
vector<int> manacher_even(const string& s) {
    int n = sz(s);
    vector<int> d2(n);
    int l = 0, r = -1;
    for (int i = 0; i < n; i++) {
        int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (0 <= i - k - 1 && i + k < n && s[i - k - 1] == s[i + k]) k++;
        d2[i] = k;
        if (i + k - 1 > r) { l = i - k; r = i + k - 1; }
    }
    return d2;
}

int longest_palindrome(const string& s) {
    auto d1 = manacher_odd(s), d2 = manacher_even(s);
    int ans = 0;
    for (int x : d1) ans = max(ans, 2 * x - 1);
    for (int x : d2) ans = max(ans, 2 * x);
    return ans;
}
```

Use: the total number of palindromic substrings is $\sum d1[i] + \sum d2[i]$.

■ String Hashing $\mathcal{O}(n \text{ build, } 1 \text{ per substring})$

```
struct Hash {
    static const ll M = (1LL << 61) - 1; // Mersenne prime
    ll B; // random base
    vector<ll> h, p;

    static ll mul(ll a, ll b) {
        __int128 c = (__int128)a * b;
        ll lo = (ll)(c & M), hi = (ll)(c >> 61);
        lo += hi;
        return lo >= M ? lo - M : lo;
    }

    Hash(const string& s) {
```

```

    B = uniform_int_distribution<ll>(256, M - 2)(rng);
    int n = sz(s);
    h.assign(n + 1, 0);
    p.assign(n + 1, 1);
    for (int i = 0; i < n; i++) {
        h[i + 1] = (mul(h[i], B) + s[i]) % M;
        p[i + 1] = mul(p[i], B);
    }

    // Hash of the inclusive range [l, r]
    ll get(int l, int r) {
        ll x = (h[r + 1] - mul(h[l], p[r - 1 + 1])) % M;
        return x < 0 ? x + M : x;
    }
};

```

Use: $O(1)$ substring comparison, so binary searching the longest common prefix of two suffixes costs $O(\log n)$.

Careful: Randomise the base. A fixed base with a small modulus is trivially hacked on Codeforces. Modulo $2^{61} - 1$ with a random base is safe in practice.

■ Trie $O(|s| \text{ per operation})$

```

struct Trie {
    static const int A = 26;
    vector<array<int, A>> nxt;
    vector<int> cnt;
    // words ending here
};

```

```

Trie() { new_node(); }

int new_node() {
    nxt.push_back({});
    nxt.back().fill(-1);
    cnt.push_back(0);
    return sz(nxt) - 1;
}

void insert(const string& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (nxt[v][c] == -1) nxt[v][c] = new_node();
        v = nxt[v][c];
    }
    cnt[v]++;
}

bool contains(const string& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (nxt[v][c] == -1) return false;
        v = nxt[v][c];
    }
    return cnt[v] > 0;
}
};

```

Use: prefix queries, autocomplete, and — with $A = 2$ over the bits of each number — maximum XOR pair queries.

7 Graphs: Traversal and Ordering

■ Representations

```

const int MAXN = 2e5 + 5;

// Adjacency list: the default choice
vector<int> adj[MAXN];
adj[u].push_back(v);
adj[v].push_back(u); // omit if directed

// Weighted
vector<pii> wadj[MAXN]; // {neighbour, weight}
wadj[u].push_back({v, w});

// Edge list: for Kruskal and Bellman-Ford
vector<tuple<int, int, int>> edges; // {w, u, v}

```

■ Graph Identities

- Tree on n nodes: exactly $n - 1$ edges, connected, acyclic.
- Complete graph K_n : $\frac{n(n-1)}{2}$ edges.
- Handshake lemma: $\sum_v \deg(v) = 2|E|$.

- A connected graph has an Euler circuit iff every degree is even; an Euler path iff exactly two degrees are odd.
- A graph is bipartite iff it has no odd-length cycle.

■ DFS $O(V + E)$

```

bool vis[MAXN];

void dfs(int u) {
    vis[u] = true;
    for (int v : adj[u])
        if (!vis[v]) dfs(v);
}

```

Careful: Recursion depth equals the longest path. For $n = 10^5$ on a path graph this overflows the default stack. Use the iterative form below.

■ Iterative DFS

```

void dfs_iter(int s) {
    vector<int> st{s};
}

```

```

vis[s] = true;
while (!st.empty()) {
    int u = st.back(); st.pop_back();
    for (int v : adj[u])
        if (!vis[v]) { vis[v] = true; st.push_back(v); }
}

```

Tip: Mark nodes visited when you *push* them, not when you pop, or the same node lands on the stack many times.

■ BFS and Distances $\mathcal{O}(V + E)$

```

vector<int> bfs(int s, int n) {
    vector<int> dist(n + 1, -1);
    queue<int> q;
    q.push(s);
    dist[s] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u])
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
    }
    return dist;
}

```

Use: shortest paths on *unweighted* graphs. `dist[v] == -1` doubles as the visited flag.

■ Multi-Source BFS

```

// Distance from the NEAREST source, in one pass
queue<int> q;
for (int s : sources) { dist[s] = 0; q.push(s); }
// ... identical BFS loop from here

```

Use: "distance to the closest fire / gate / water cell". Seeding every source at once beats running one BFS per source.

■ 0-1 BFS $\mathcal{O}(V + E)$

```

// Shortest path when every edge weight is 0 or 1
deque<int> dq;
vector<int> dist(n, INT_MAX);
dist[s] = 0;
dq.push_front(s);
while (!dq.empty()) {
    int u = dq.front(); dq.pop_front();
    for (auto [v, w] : wadj[u]) {
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            if (w == 0) dq.push_front(v);
            else dq.push_back(v);
        }
    }
}

```

Use: a Dijkstra replacement without the log factor when the only weights are 0 and 1.

■ Grid Traversal

```

int dx[4] = {-1, 1, 0, 0};
int dy[4] = {0, 0, -1, 1};

// 8 directions, including diagonals
int ex[8] = {-1,-1,-1, 0, 0, 1, 1, 1};
int ey[8] = {-1, 0, 1,-1, 1,-1, 0, 1};

for (int k = 0; k < 4; k++) {
    int nx = x + dx[k], ny = y + dy[k];
    if (nx < 0 || nx >= R || ny < 0 || ny >= C) continue;
    if (grid[nx][ny] == '#') continue;
    // ...
}

```

Tip: Flatten a cell to `x * C + y` when you need it as a single integer id (for a DSU, or a `dist` array).

■ Connected Components $\mathcal{O}(V + E)$

```

int components = 0;
for (int i = 1; i <= n; i++)
    if (!vis[i]) { components++; dfs(i); }

```

■ Bipartite Check $\mathcal{O}(V + E)$

```

vector<int> color(MAXN, -1);

bool bipartite(int n) {
    for (int s = 1; s <= n; s++) {
        if (color[s] != -1) continue;
        color[s] = 0;
        queue<int> q;
        q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) {
                if (color[v] == -1) {
                    color[v] = color[u] ^ 1;
                    q.push(v);
                } else if (color[v] == color[u]) {
                    return false;
                }
            }
        }
    }
    return true;
}

```

Careful: Loop over every start node. Checking only node 1 misses disconnected components.

■ Cycle in an Undirected Graph $\mathcal{O}(V + E)$

```

bool dfs_cycle(int u, int parent) {

```

```

vis[u] = true;
for (int v : adj[u]) {
    if (!vis[v]) {
        if (dfs_cycle(v, u)) return true;
    } else if (v != parent) {
        return true;
    }
}
return false;
}

```

Careful: With multi-edges, comparing to `parent` by node id is wrong — track the incoming edge id instead.

■ Cycle in a Directed Graph $\mathcal{O}(V + E)$

```

// 0 = unvisited, 1 = on the current path, 2 = done
int state[MAXN];

bool dfs_cycle_dir(int u) {
    state[u] = 1;
    for (int v : adj[u]) {
        if (state[v] == 1) return true; // back edge
        if (state[v] == 0 && dfs_cycle_dir(v)) return true;
    }
    state[u] = 2;
    return false;
}

```

Careful: State 2 matters: revisiting a *finished* node is a cross edge, not a cycle. A plain `vis` array reports false cycles.

■ Topological Sort (Kahn) $\mathcal{O}(V + E)$

```

vector<int> topo_sort(int n) {
    vector<int> indeg(n + 1, 0), order;
    for (int u = 1; u <= n; u++)
        for (int v : adj[u]) indeg[v]++;

    queue<int> q; // priority_queue for
    for (int i = 1; i <= n; i++) // lexicographically smallest
        if (indeg[i] == 0) q.push(i);

    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u])
            if (--indeg[v] == 0) q.push(v);
    }
    if (sz(order) != n) return {}; // a cycle exists
    return order;
}

```

Use: a short result means the graph has a cycle — this doubles as cycle detection. Longest path on a DAG is a DP over this order.

■ Topological Sort (DFS)

```

void topo_dfs(int u, vector<int>& order) {
    vis[u] = true;
    for (int v : adj[u]) if (!vis[v]) topo_dfs(v, order);
    order.push_back(u); // post-order
}
// Reverse `order` at the end

```

8 Shortest Paths, MST and DSU

■ Dijkstra $\mathcal{O}(E \log V)$

```

vector<ll> dijkstra(int s, int n) {
    vector<ll> dist(n + 1, INF);
    priority_queue<pll, vector<pll>, greater<pll>> pq;
    dist[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue; // stale entry

        for (auto [v, w] : wadj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}

```

Careful: Fails with negative weights. The `if (d > dist[u]) continue` line is not optional — without it, stale queue entries reprocess nodes and the complexity degrades.

■ Path Reconstruction

```

vector<int> par(n + 1, -1);
// inside the relaxation:
// par[v] = u;

vector<int> path;
for (int cur = t; cur != -1; cur = par[cur])
    path.push_back(cur);
reverse(all(path));

```

■ Bellman-Ford $\mathcal{O}(VE)$

```

// Handles negative weights and detects negative cycles
vector<ll> bellman_ford(int s, int n, bool& has_neg_cycle) {

```

```

vector<ll> dist(n + 1, INF);
dist[s] = 0;
for (int it = 0; it < n - 1; it++)
    for (auto [w, u, v] : edges)
        if (dist[u] < INF && dist[u] + w < dist[v])
            dist[v] = dist[u] + w;

has_neg_cycle = false;
for (auto [w, u, v] : edges) // one extra pass
    if (dist[u] < INF && dist[u] + w < dist[v])
        has_neg_cycle = true;
return dist;
}

```

Use: a distance that still improves on pass n can only mean a negative cycle.

■ Floyd-Warshall $\mathcal{O}(V^3)$

```

// d[i][j] = weight of edge i->j, INF if none, 0 on diagonal
for (int k = 1; k <= n; k++)
    for (int i = 1; i <= n; i++) {
        if (d[i][k] == INF) continue; // big speedup
        for (int j = 1; j <= n; j++)
            if (d[i][k] + d[k][j] < d[i][j])
                d[i][j] = d[i][k] + d[k][j];
    }

// d[i][i] < 0 => i lies on a negative cycle

```

Careful: k must be the outermost loop. Any other order silently computes the wrong answer. Practical up to $n \approx 500$.

Tip: With d as booleans and $|\neq \&$ instead of $+\min$, the same loop is transitive closure (reachability).

■ DSU / Union-Find $\mathcal{O}(\alpha(n))$

```

struct DSU {
    vector<int> parent, sz;
    int components;

    DSU(int n) : parent(n + 1), sz(n + 1, 1), components(n) {
        iota(all(parent), 0);
    }

    int find(int x) { // path compression
        return parent[x] == x
            ? x
            : parent[x] = find(parent[x]);
    }

    bool unite(int a, int b) { // union by size
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
    }
}

```

```

parent[b] = a;
sz[a] += sz[b];
components--;
return true;
}

bool same(int a, int b) { return find(a) == find(b); }
int size(int x) { return sz[find(x)]; }
};

```

Use: connectivity under edge insertions, Kruskal, and counting components online. $\alpha(n) \leq 4$ for any realistic n — treat it as constant.

Careful: DSU cannot delete edges. If the problem removes edges, process the queries in reverse.

■ Kruskal MST $\mathcal{O}(E \log E)$

```

ll kruskal(int n, vector<tuple<int,int,int>>& edges) {
    sort(all(edges)); // by weight
    DSU dsu(n);
    ll total = 0;
    int used = 0;
    for (auto [w, u, v] : edges) {
        if (dsu.unite(u, v)) {
            total += w;
            if (++used == n - 1) break;
        }
    }
    return used == n - 1 ? total : -1; // -1: disconnected
}

```

Tip: Sort descending for a maximum spanning tree. For the "minimise the largest edge on a path" family, the MST is the answer.

■ Prim MST $\mathcal{O}(E \log V)$

```

// Better than Kruskal on dense graphs
ll prim(int n) {
    vector<bool> in(n + 1, false);
    priority_queue<pll, vector<pll>, greater<pll>> pq;
    pq.push({0, 1});
    ll total = 0;
    int used = 0;
    while (!pq.empty() && used < n) {
        auto [w, u] = pq.top(); pq.pop();
        if (in[u]) continue;
        in[u] = true;
        total += w;
        used++;
        for (auto [v, c] : wadj[u])
            if (!in[v]) pq.push({c, v});
    }
    return used == n ? total : -1;
}

```

9 Trees and Advanced Graphs

■ Rooting: Parent, Depth, Subtree Size $\mathcal{O}(n)$

```
vector<int> par, dep, sub;

void root_tree(int u, int p) {
    par[u] = p;
    sub[u] = 1;
    for (int v : adj[u]) {
        if (v == p) continue;
        dep[v] = dep[u] + 1;
        root_tree(v, u);
        sub[u] += sub[v];
    }
}
// call: dep[1] = 0; root_tree(1, 0);
```

Tip: Skipping the parent is all a tree DFS needs — no visited array.

■ Tree Diameter $\mathcal{O}(n)$

```
// Two BFS passes. Works on unweighted trees.
// 1. BFS from any node -> the farthest node a
// 2. BFS from a -> the farthest node b
// dist(a, b) is the diameter
int farthest(int s, int n) {
    auto d = bfs(s, n);
    int best = s;
    for (int i = 1; i <= n; i++)
        if (d[i] > d[best]) best = i;
    return best;
}
```

Careful: The two-BFS trick is only valid on trees (and on graphs with non-negative weights using Dijkstra). It is wrong on general graphs.

■ LCA with Binary Lifting $\mathcal{O}(n \log n \text{ build, } \log n \text{ query})$

```
const int LOG = 18; // 2^18 > 2*10^5
int up[MAXN][LOG];

void build_lca(int n) {
    for (int i = 1; i <= n; i++) up[i][0] = par[i];
    for (int j = 1; j < LOG; j++)
        for (int i = 1; i <= n; i++)
            up[i][j] = up[ up[i][j-1] ][j-1];
}

int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    int diff = dep[u] - dep[v];
    for (int j = 0; j < LOG; j++) // lift u up
        if (diff & (1 << j)) u = up[u][j];
    if (u == v) return u;

    for (int j = LOG - 1; j >= 0; j--) // lift together
        if (up[u][j] != up[v][j]) {
```

```
            u = up[u][j];
            v = up[v][j];
        }
    }
    return up[u][0];
}

int dist(int u, int v) {
    return dep[u] + dep[v] - 2 * dep[lca(u, v)];
}

int kth_ancestor(int u, int k) {
    for (int j = 0; j < LOG && u; j++)
        if (k & (1 << j)) u = up[u][j];
    return u;
}
```

Careful: Set `par[root] = root` (or 0 with `up[0][*] = 0`) so lifting past the root is harmless.

■ Euler Tour: Subtree as a Range $\mathcal{O}(n)$

```
int timer = 0;
vector<int> tin(MAXN), tout(MAXN);

void euler(int u, int p) {
    tin[u] = timer++;
    for (int v : adj[u]) if (v != p) euler(v, u);
    tout[u] = timer; // half-open [tin, tout)

    // u is an ancestor of v
    bool is_anc(int u, int v) {
        return tin[u] <= tin[v] && tout[v] <= tout[u];
    }
}
```

Use: the subtree of `u` is exactly the contiguous range $[tin_u, tout_u)$. Subtree sum and subtree update become an ordinary BIT or segment tree range operation.

■ DP on Trees $\mathcal{O}(n)$

```
// Example: largest independent set in a tree
// dp[u][0] = u not taken, dp[u][1] = u taken
11 dp[MAXN][2];

void tree_dp(int u, int p) {
    dp[u][0] = 0;
    dp[u][1] = 1;
    for (int v : adj[u]) {
        if (v == p) continue;
        tree_dp(v, u);
        dp[u][0] += max(dp[v][0], dp[v][1]);
        dp[u][1] += dp[v][0];
    }
}
// answer: max(dp[root][0], dp[root][1])
```

■ Bridges $\mathcal{O}(V + E)$

An edge whose removal disconnects the graph.

```
vector<int> tin_b(MAXN, -1), low(MAXN);
vector<pii> bridges;
int timer_b = 0;

void dfs_bridge(int u, int pe) { // pe = incoming edge id
    tin_b[u] = low[u] = timer_b++;
    for (auto [v, id] : adj_id[u]) {
        if (id == pe) continue;
        if (tin_b[v] != -1) {
            low[u] = min(low[u], tin_b[v]);
        } else {
            dfs_bridge(v, id);
            low[u] = min(low[u], low[v]);
            if (low[v] > tin_b[u]) bridges.push_back({u, v});
        }
    }
}
```

Careful: Skip by *edge id*, not by parent node — otherwise parallel edges are misreported as bridges. `adj_id[u]` stores {neighbour, edge_id}.

■ Articulation Points $\mathcal{O}(V + E)$

```
vector<bool> is_art(MAXN, false);

void dfs_art(int u, int p) {
    tin_b[u] = low[u] = timer_b++;
    int children = 0;
    for (int v : adj[u]) {
        if (v == p) continue;
        if (tin_b[v] != -1) {
            low[u] = min(low[u], tin_b[v]);
        } else {
            dfs_art(v, u);
            low[u] = min(low[u], low[v]);
        }
    }
}
```

```
        if (low[v] >= tin_b[u] && p != -1) is_art[u] = true;
        children++;
    }
    if (p == -1 && children > 1) is_art[u] = true;
}
```

Careful: The root is special: it is an articulation point only when it has more than one DFS child.

■ SCC (Kosaraju) $\mathcal{O}(V + E)$

```
vector<int> order_k, comp;
vector<int> radj[MAXN]; // reversed graph

void dfs1(int u) {
    vis[u] = true;
    for (int v : adj[u]) if (!vis[v]) dfs1(v);
    order_k.push_back(u);
}

void dfs2(int u, int c) {
    comp[u] = c;
    for (int v : radj[u]) if (comp[v] == -1) dfs2(v, c);
}

int kosaraju(int n) {
    comp.assign(n + 1, -1);
    for (int i = 1; i <= n; i++) if (!vis[i]) dfs1(i);
    reverse(all(order_k));
    int c = 0;
    for (int u : order_k)
        if (comp[u] == -1) dfs2(u, c++);
    return c; // number of SCCs
}
```

Use: contracting each SCC to a single node yields a DAG (the condensation), which you can then topologically sort or DP over.

10 Dynamic Programming

■ How to Build a DP

1. **State** — what exactly does `dp[i][j]` mean? Write it down as a sentence first.
2. **Transition** — how do smaller states combine into it?
3. **Base case** — the smallest state, filled directly.
4. **Order** — iterate so every dependency is already computed.
5. **Answer** — which state (or which max/sum over states) is the result?

Tip: Total cost = number of states $\times$ cost per transition. Check that against the constraints before writing any code.

■ 0/1 Knapsack $\mathcal{O}(nW)$

```
// Each item used at most once
vector<ll> dp(W + 1, 0);
for (int i = 0; i < n; i++)
    for (int w = W; w >= wt[i]; w--) // DESCENDING
        dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
// dp[W] = best value within capacity W
```

Careful: The descending loop is what enforces "at most once". Ascending turns it into the unbounded version.

■ Unbounded Knapsack / Coin Change $\mathcal{O}(nW)$

```
// Minimum number of coins summing to W
vector<int> dp(W + 1, IINF);
dp[0] = 0;
for (int c : coins)
    for (int w = c; w <= W; w++) // ASCENDING
        if (dp[w - c] != IINF)
            dp[w] = min(dp[w], dp[w - c] + 1);

// Number of distinct combinations (order ignored)
vector<ll> ways(W + 1, 0);
ways[0] = 1;
for (int c : coins)
    for (int w = c; w <= W; w++)
        ways[w] += ways[w - c];
```

Careful: Loop order decides the meaning. Coins outside, sums inside counts *combinations*; sums outside, coins inside counts *permutations*.

■ LCS $\mathcal{O}(nm)$

```
vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        dp[i][j] = (a[i-1] == b[j-1])
            ? dp[i-1][j-1] + 1
            : max(dp[i-1][j], dp[i][j-1]);
```

■ Edit Distance $\mathcal{O}(nm)$

```
vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
for (int i = 0; i <= n; i++) dp[i][0] = i; // deletions
for (int j = 0; j <= m; j++) dp[0][j] = j; // insertions

for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        dp[i][j] = (a[i-1] == b[j-1])
            ? dp[i-1][j-1]
            : 1 + min({dp[i-1][j], // delete
                      dp[i][j-1], // insert
                      dp[i-1][j-1]}); // replace
```

■ Bitmask DP (TSP) $\mathcal{O}(2^n n^2)$

```
// dp[mask][i] = min cost visiting `mask`, standing at i
vector<vector<ll>> dp(1 << n, vector<ll>(n, INF));
dp[1][0] = 0; // start at node 0

for (int mask = 1; mask < (1 << n); mask++)
    for (int i = 0; i < n; i++) {
        if (dp[mask][i] == INF) continue;
        if (!(mask & (1 << i))) continue;
        for (int j = 0; j < n; j++) {
            if (mask & (1 << j)) continue;
            int nm = mask | (1 << j);
            dp[nm][j] = min(dp[nm][j], dp[mask][i] + cost[i][j]);
        }
    }
```

```
ll ans = INF;
for (int i = 1; i < n; i++)
    ans = min(ans, dp[(1 << n) - 1][i] + cost[i][0]);
```

Use: $n \leq 20$ is the giveaway that a bitmask DP is intended.

■ Digit DP $\mathcal{O}(|d| \cdot 10 \cdot \text{states})$

```
string num;
ll memo[20][2][200];
bool seen[20][2][200];

// Counts values in [0, num] with some digit property
ll go(int pos, bool tight, int acc) {
    if (pos == sz(num)) return good(acc) ? 1 : 0;
    if (!tight && seen[pos][tight][acc])
        return memo[pos][tight][acc];

    int lim = tight ? num[pos] - '0' : 9;
    ll res = 0;
    for (int d = 0; d <= lim; d++)
        res += go(pos + 1, tight && (d == lim), acc + d);

    if (!tight) {
        seen[pos][tight][acc] = true;
        memo[pos][tight][acc] = res;
    }
    return res;
}
// Range [L, R]: go(R) - go(L - 1)
```

Careful: Only memoise the `!tight` states — tight states depend on the specific prefix and are not reusable.

■ Longest Path on a DAG $\mathcal{O}(V + E)$

```
// Process nodes in topological order
vector<ll> dp(n + 1, 0);
for (int u : topo_order)
    for (int v : adj[u])
        dp[v] = max(dp[v], dp[u] + 1);
```

Use: longest path is NP-hard on general graphs but linear on a DAG, because the topological order removes all cyclic dependencies.

■ Memory Optimisation

```
// When dp[i] only reads dp[i-1], keep two rows
vector<int> prev(m + 1, 0), cur(m + 1, 0);
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) cur[j] = /* ... */;
    swap(prev, cur);
}
```

Tip: $10^5 \times 10^5$ ints is 40 GB; two rows is 800 KB. Rolling arrays are often the difference between MLE and AC.

11 Advanced Data Structures

Fenwick Tree (BIT) $\mathcal{O}(\log n)$

```
struct Fenwick {
    int n;
    vector<ll> bit;

    Fenwick(int n) : n(n), bit(n + 1, 0) {}

    void add(int i, ll v) {          // 1-indexed
        for (; i <= n; i += i & -i) bit[i] += v;
    }

    ll sum(int i) {                  // prefix [1, i]
        ll r = 0;
        for (; i > 0; i -= i & -i) r += bit[i];
        return r;
    }

    ll range(int l, int r) { return sum(r) - sum(l - 1); }

    // Smallest i with prefix sum >= target,  $\mathcal{O}(\log n)$ 
    int lower_bound(ll target) {
        int pos = 0;
        for (int pw = 1 << __lg(n); pw; pw >>= 1)
            if (pos + pw <= n && bit[pos + pw] < target) {
                pos += pw;
                target -= bit[pos];
            }
        return pos + 1;
    }
};
```

Careful: A BIT is 1-indexed. Feeding it index 0 makes `add` loop forever, because `0 & -0 == 0`.

Tip: Smaller and roughly twice as fast as a segment tree. Reach for a BIT whenever prefix sums are enough.

Fenwick: Range Update, Point Query $\mathcal{O}(\log n)$

```
// Store the difference array inside the BIT
void add_range(Fenwick& f, int l, int r, ll v) {
    f.add(l, v);
    f.add(r + 1, -v);
}
ll point_query(Fenwick& f, int i) { return f.sum(i); }
```

2D Fenwick $\mathcal{O}(\log^2 n)$

```
ll bit2[N + 1][M + 1];

void add2(int x, int y, ll v) {
    for (int i = x; i <= N; i += i & -i)
        for (int j = y; j <= M; j += j & -j)
            bit2[i][j] += v;
}
```

```
ll sum2(int x, int y) {
    ll r = 0;
    for (int i = x; i > 0; i -= i & -i)
        for (int j = y; j > 0; j -= j & -j)
            r += bit2[i][j];
    return r;
}

ll rect2(int x1, int y1, int x2, int y2) {
    return sum2(x2, y2) - sum2(x1 - 1, y2)
        - sum2(x2, y1 - 1) + sum2(x1 - 1, y1 - 1);
}
```

Segment Tree (Recursive) $\mathcal{O}(\log n)$

```
const int MAXN = 2e5 + 5;
ll t[4 * MAXN];
int a[MAXN];

void build(int v, int tl, int tr) {
    if (tl == tr) { t[v] = a[tl]; return; }
    int tm = (tl + tr) / 2;
    build(2*v, tl, tm);
    build(2*v + 1, tm + 1, tr);
    t[v] = t[2*v] + t[2*v + 1];
}

ll query(int v, int tl, int tr, int l, int r) {
    if (l > r) return 0; // neutral element
    if (l == tl && r == tr) return t[v];
    int tm = (tl + tr) / 2;
    return query(2*v, tl, tm, l, min(r, tm))
        + query(2*v + 1, tm + 1, tr, max(l, tm + 1), r);
}

void update(int v, int tl, int tr, int pos, ll val) {
    if (tl == tr) { t[v] = val; return; }
    int tm = (tl + tr) / 2;
    if (pos <= tm) update(2*v, tl, tm, pos, val);
    else update(2*v + 1, tm + 1, tr, pos, val);
    t[v] = t[2*v] + t[2*v + 1];
}
```

Tip: For min/max, swap + for min/max and change the neutral element to `INF/-INF`. Size the array `4n`, not `2n`.

Segment Tree (Iterative) $\mathcal{O}(\log n)$

```
// Shorter, ~2x faster, point update + range query
int N;
ll t[2 * MAXN];

void build_it(int n) {
    N = n;
    for (int i = N - 1; i > 0; i--) t[i] = t[2*i] + t[2*i+1];
}
```

```

void update_it(int p, ll val) {
    for (t[p += N] = val; p > 1; p >>= 1)
        t[p >> 1] = t[p] + t[p ^ 1];
}

ll query_it(int l, int r) {          // inclusive [l, r]
    ll res = 0;
    for (l += N, r += N + 1; l < r; l >>= 1, r >>= 1) {
        if (l & 1) res += t[l++];
        if (r & 1) res += t[--r];
    }
    return res;
}

```

Careful: Leaves live at $t[N \dots 2N-1]$. Fill them before calling `build_it`.

■ Segment Tree with Lazy Propagation $\mathcal{O}(\log n)$

```

ll t[4 * MAXN], lz[4 * MAXN];

void push(int v, int tl, int tr) {
    if (!lz[v]) return;
    t[v] += lz[v] * (tr - tl + 1);
    if (tl != tr) {
        lz[2*v] += lz[v];
        lz[2*v + 1] += lz[v];
    }
    lz[v] = 0;
}

void update(int v, int tl, int tr, int l, int r, ll val) {
    push(v, tl, tr);
    if (l > tr || tl > r) return;
    if (l <= tl && tr <= r) {
        lz[v] += val;
        push(v, tl, tr);
        return;
    }
    int tm = (tl + tr) / 2;
    update(2*v, tl, tm, l, r, val);
    update(2*v + 1, tm + 1, tr, l, r, val);
    t[v] = t[2*v] + t[2*v + 1];
}

ll query(int v, int tl, int tr, int l, int r) {
    push(v, tl, tr);
    if (l > tr || tl > r) return 0;
    if (l <= tl && tr <= r) return t[v];
    int tm = (tl + tr) / 2;
    return query(2*v, tl, tm, l, r)
        + query(2*v + 1, tm + 1, tr, l, r);
}

```

Careful: `push` must run at the top of *every* recursive call, including queries. A forgotten push is the classic lazy-propagation bug.

■ Sparse Table (RMQ) $\mathcal{O}(n \log n \text{ build, } 1 \text{ query})$

```

int sp[LOG][MAXN], lg[MAXN];

void build_sparse(int n) {
    lg[1] = 0;
    for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;

    for (int i = 0; i < n; i++) sp[0][i] = a[i];
    for (int j = 1; j < LOG; j++)
        for (int i = 0; i + (1 << j) <= n; i++)
            sp[j][i] = min(sp[j-1][i],
                           sp[j-1][i + (1 << (j-1))]);
}

int query_min(int l, int r) {          // inclusive
    int j = lg[r - l + 1];
    return min(sp[j][l], sp[j][r - (1 << j) + 1]);
}

```

Careful: Only valid for *idempotent* operations — min, max, gcd — where overlapping ranges are harmless. It does not work for sums, and it does not support updates.

■ PBDS Ordered Set $\mathcal{O}(\log n)$

```

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

typedef tree<int, null_type, less<int>, rb_tree_tag,
            tree_order_statistics_node_update> ordered_set;

ordered_set s;
s.insert(1);
s.insert(9);

// k-th smallest, 0-indexed
auto it = s.find_by_order(0);
// how many elements are strictly smaller than x
int cnt = s.order_of_key(5);

```

Careful: GCC only. It rejects duplicates like a `set`; for a multiset use `less_equal<int>` as the comparator — but then `erase(value)` stops working and you must erase by iterator.

12 Contest Techniques and Pitfalls

■ Meet in the Middle $\mathcal{O}(2^{n/2} \cdot n)$

```

// Subset sums for n <= 40: split into two halves
vector<ll> gen(vector<ll>& v) {

```

```

    int k = sz(v);
    vector<ll> sums;
    for (int mask = 0; mask < (1 << k); mask++) {
        ll s = 0;
        for (int i = 0; i < k; i++)

```

```

        if (mask & (1 << i)) s += v[i];
        sums.push_back(s);
    }
    return sums;
}
// Sort one half, binary search it for each element
// of the other half.

```

Use: $n \leq 40$ where 2^n is hopeless but 2^{20} is instant.

■ Interactive Problems

```

// Flush after every query or you will deadlock
cout << "? " << x << endl; // endl flushes
cin >> response;

cout << "! " << answer << endl;

```

Careful: This is the one place where `endl` is correct and `'\n'` is a bug.

■ Stress Testing

When a solution fails on a hidden test, generate your own:

```

# gen.py writes a small random test to stdout
for i in $(seq 1 1000); do
    python gen.py $i > in.txt
    ./fast < in.txt > out1.txt
    ./brute < in.txt > out2.txt
    if ! diff -q out1.txt out2.txt > /dev/null; then
        echo "Mismatch on test $i"
        cat in.txt
        break
    fi
done

```

Tip: Generate *small* tests ($n \leq 8$). A minimal counterexample you can read beats a 200-line one you cannot.

■ Greedy: Proving It Works

- **Exchange argument** — show any optimal solution can be rewritten into the greedy one without getting worse.
- **Stays ahead** — show the greedy prefix is at least as good as any other prefix at every step.
- If neither works, it is probably a DP.

Tip: Cheapest check: brute-force small cases and diff against your greedy. A counterexample appears in seconds if there is one.

■ Reading the Problem

- Is the answer guaranteed to exist, or must you print -1 ?
- How many test cases per file? Are the sums of n bounded?
- Are indices 0-based or 1-based in the statement?
- Can $n = 0$ or $n = 1$? Can the graph be disconnected?
- Are values negative? Do they overflow `int`?
- Is the graph a tree, a DAG, or fully general?

■ Bug Checklist

Read this before rewriting anything from scratch.

- `int` overflow: $(1l)a * b$, not $a * b$.
- Arrays not reset between test cases.
- Off-by-one: inclusive vs half-open ranges.
- 0-indexed code fed 1-indexed input.
- $n == 1$, empty input, all-equal values.
- Recursion depth on a path graph.
- `cin` left a newline before `getline`.
- Comparator not a strict weak ordering.
- `endl` inside a tight output loop (slow).

Careful: Clearing globals with `memset` across 10^5 test cases is itself $O(t \cdot n)$. Reset only the indices you actually touched.

■ Time Management

- Read *every* problem before starting. The easiest is rarely problem A.
- Sort by how many teams have solved it, when the scoreboard shows that.
- Stuck for 20 minutes with no idea? Switch problems and come back.
- Write the brute force first when the statement is confusing — it clarifies the problem and doubles as a stress-test oracle.